Homework X2: MLP for MNIST Classification

Introduction to Deep Learning

Optional | Maximum: 0 points | Not counted toward the final grade

Deadline: October 26, 2026 (Monday), 12:00 noon

Overview and learning goals

Extend Homework 1's linear MNIST classifier to a one-hidden-layer multilayer perceptron (MLP). Implement the forward pass, stable softmax cross-entropy, backpropagation, and mini-batch SGD using NumPy arrays. The notebook marks five `TODO` blocks; `mnist_data_loader.py` reads the supplied local MNIST files.

MNIST has 60,000 training and 10,000 test images; each has $28 \times 28 = 784$ pixels. Use the same NumPy seed and split recipe as Homework 1: 50,000 images for training, 10,000 for validation, and the separate 10,000-image test set for final evaluation. Scale pixels to $[0, 1]$. The four compressed files are provided in `mnist_data/`; no download is needed.

Model and notation

For one image $x \in \mathbb{R}^{784}$ with pixels scaled to $[0, 1]$, use H hidden units and ten output classes:

$$a_h = b_h^{(1)} + \sum_{j=1}^{784} W_{hj}^{(1)} x_j, \qquad q_h = \max(0, a_h), \tag{1}$$

$$z_k = b_k^{(2)} + \sum_{h=1}^{H} W_{kh}^{(2)} q_h, \qquad p_k = \frac{\exp(z_k - m)}{\sum_{r=0}^{9} \exp(z_r - m)}, \quad m = \max_r z_r. \tag{2}$$

Thus $W^{(1)} \in \mathbb{R}^{H \times 784}$, $b^{(1)} \in \mathbb{R}^H$, $W^{(2)} \in \mathbb{R}^{10 \times H}$, and $b^{(2)} \in \mathbb{R}^{10}$. For a mini-batch $\mathcal{B}$ of size B, minimize

$$L = -\frac{1}{B} \sum_{i \in \mathcal{B}} \log p_{i,y_i} + \frac{\lambda}{2} \left(\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2 \right). \tag{3}$$

Regularize the two weight matrices, but not the biases. Use the same objective in your derivation, code, and report. Set the ReLU derivative at zero to 0. The predicted class is $\hat{y} = \arg\max_k z_k$.

Tasks (optional; no graded points)

1. **Derive backpropagation.** In your report, derive the gradients for all four parameter groups in Eq. (3). State their shapes and explain the role of the ReLU mask. Explain why subtracting the largest logit preserves the softmax probabilities and how to compute the negative log probability without taking log(0).

2. **Implement the MLP from scratch.** Complete the notebook's five marked `TODO` blocks. Use NumPy array operations for examples and classes; loops over epochs and mini-batches are allowed. Do not call prebuilt neural-network, gradient, softmax, or cross-entropy routines. Make the supplied shape, extreme-logit, and finite-difference checks pass.

3. **Train and validate.** Use the full fixed split and seed. Train at least two learning rates while holding other settings fixed, and at least two hidden-layer widths while holding other settings fixed. Plot training loss and training/validation accuracy by epoch. Report each run's settings and final training and validation accuracy.

4. **Evaluate and discuss.** Choose one configuration using validation accuracy, then evaluate it once on the separate test set. Report the chosen configuration and test accuracy. In 4–6 sentences, discuss the learning-rate and width comparisons, any sign of underfitting or overfitting, and one difference from the linear model in Homework 1.

Rules and submission

- This is an **optional assignment with a maximum of 0 points**; it does not contribute to the final course grade.
- Submit the completed `HWX2_MLP_MNIST.ipynb` with all cells executed, the supplied loader, and a short PDF report with your derivation, results table, curves, and discussion. Put your name and student ID in the report. Do not submit the MNIST data files.
- Use Python 3, **NumPy**, Matplotlib, and the Python standard library. Implement the MLP, loss, gradients, and optimizer yourself. Do not use PyTorch, TensorFlow, JAX, scikit-learn neural networks, automatic differentiation, or prebuilt model/loss/optimizer routines.
- Do not update parameters with validation or test examples, and do not tune hyperparameters using the test set. Keep the random seed and split so the experiment is reproducible. Work independently and cite external material you use.